CSE 213

Software Requirement Analysis

Why Requirement Analysis Matters

Poor Requirement analysis is one of the major reasons of Project Failure

- **Misaligned Features:** Teams build the wrong or misaligned features. This often causes total rewrites.
- **Architectural Churn:** Constant changes break code structure, leads to bugs and often hampers architectures in the mid-way.
- **Resource Drain:** Mid-flight revisions cause missed deadlines, budget overruns, and team burnout.



Cost Reduction

Fixing errors early costs much less than post-release hotfixes.



Stakeholder Alignment

Establishes a single source of truth across product teams.

The Analysis Pipeline

STAGE 01

Elicitation

Gather raw user needs and domain constraints.

STAGE 02

Analysis & Refinement

Resolve conflicts and model system behavior.

STAGE 03

Specification

Document precise formal criteria

STAGE 04

Validation

Verify specifications against stakeholder objectives.

Primary Elicitation Methods

1-on-1 Interviews

In-depth qualitative insights from domain experts and key users.

Quantitative Surveys

Structured data gathering across broad user bases for statistical validation.

Focus Groups

Interactive sessions uncovering shared priorities and group consensus.

Primary Elicitation Methods (Contd.)



Co-Design Workshops

Joint creation sessions where stakeholders design solutions collaboratively.



Observational/Field Study

Observing user workflows in their natural operational environment.

Primary Elicitation Methods (Contd.)

Technique	Best for	Watch out for
Interviews	Deep needs, domain rules	Interviewer bias, Scale issue
Observation / field study	Real workflows, workarounds	Time-consuming; observer effect
Surveys / questionnaires	Many users, quantifiable views	Shallow answers; low response
Workshops	Conflicting stakeholders, quick consensus	Dominant voices; scheduling issue

TAXONOMY

Functional vs. Non-Functional Requirements

Functional

What the system MUST DO

Send money, check balance, cash-in via agent, view transaction history.

Non-Functional

How WELL the system performs

Works offline, loads in 3 s on 3G, usable by low-literacy users, audit logging.

System Representation Techniques

Use Case Diagrams

Maps actor interactions and functional system boundaries.

BPMN & Data Flow Diagrams

Illustrates data flow and process transformations.

UML Class Diagrams

Defines structural domain entities and relationships.

Prioritization Framework

Need to prioritize some requirements over others. Cannot do all or do all at the same time or give equal importance to all.

The MoSCoW Method

MUST HAVE
Non-negotiable core functionality required for release.

SHOULD HAVE
High-value features with viable workarounds if omitted.

COULD HAVE
Desirable enhancements included only if resources permit.

WON'T HAVE
Out of scope for current development cycle.

Many other methods also exist.

Anatomy of an Effective Requirement

Unambiguous	**Verifiable**	**Complete**
Consistent	**Measurable**	**Feasible**

Weak

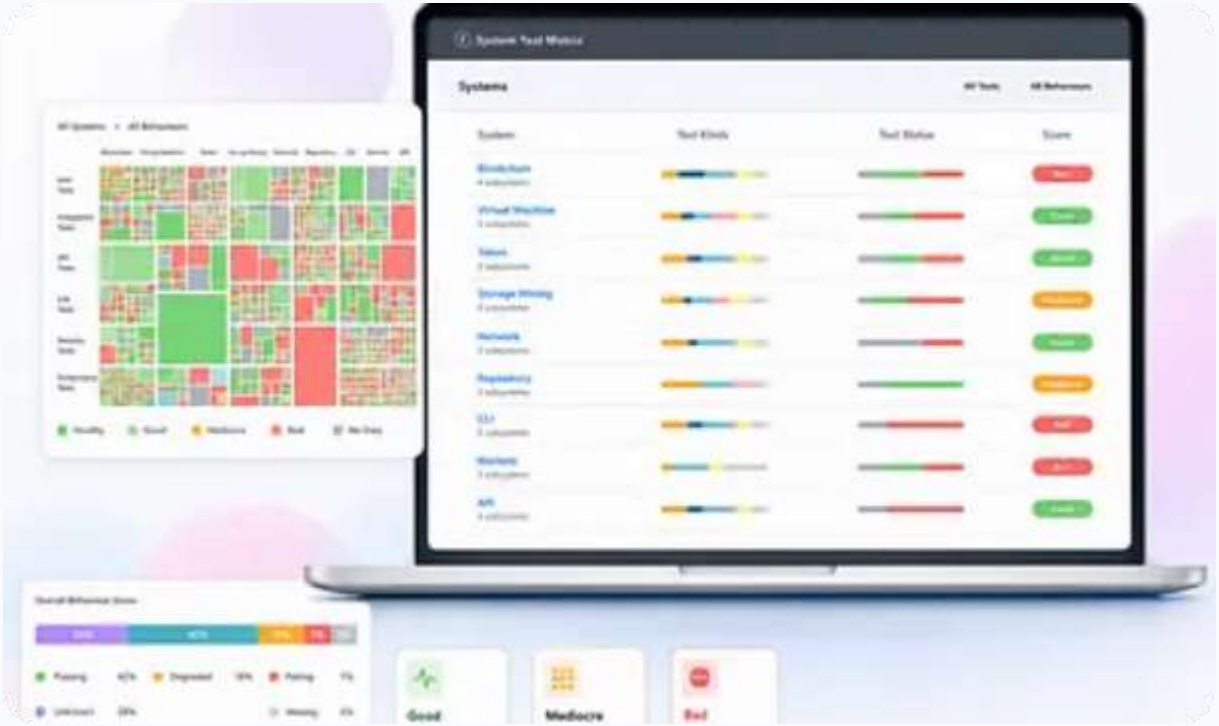
“The app should be fast and easy to use for rural users.”

Testable

“A balance check completes in ≤ 3 s on a 3G link for 95% of requests; 8 of 10 first-time users send money unaided in a pilot test.”

Your task is translate the ambiguous, vague requirements received from the user to clear, measurable ones.

Requirements Traceability Matrix (RTM)



Bi-Directional Tracing

Links business goals forward to test cases and backward to origin.

Impact Analysis

Evaluates change scope prior to approving modifications.

Verification Coverage

Ensures zero orphan features or missing test plans.

Precision in Analysis Drives Excellence in Code

Questions & Discussion